

Memoria del Proyecto: Aplicacion Meteorologica con React y AEMET

Autor: Jose Francisco Martinez Costas
Modulo: Desarrollo en Entorno Cliente (DWEC)
Curso: 2o Desarrollo de Aplicaciones Web
Centro: CIFP Carlos III - Cartagena
Fecha: Junio 2026

Indice

- 1 Descripcion del proyecto
 - 2 Objetivos
 - 3 Tecnologias utilizadas
 - 4 Arquitectura del sistema
 - 5 Estructura del repositorio
 - 6 Backend: servidor Express y AEMET
 - 7 Frontend: aplicacion React
 - 8 Instalacion y ejecucion
 - 9 Decisiones tecnicas
 - 10 Mejoras implementadas
 - 11 Gestion de errores
 - 12 Dificultades encontradas y soluciones
 - 13 Conclusiones
 - 14 Referencias
-

1. Descripcion del proyecto

Este proyecto consiste en una **aplicacion web meteorologica** que permite consultar la prediccion del tiempo en cualquier municipio de Espana. La aplicacion obtiene los datos de la **API oficial de AEMET** (Agencia Estatal de Meteorologia) y los presenta de forma clara y comprensible al usuario.

Que problema resuelve

Muchas personas necesitan consultar el tiempo de forma rapida antes de salir de casa, planificar actividades o viajes. Esta aplicacion centraliza esa informacion en una interfaz sencilla, sin necesidad de navegar por la web de AEMET ni entender su formato tecnico.

Para quien esta pensada

La aplicacion esta dirigida a cualquier usuario que quiera consultar el tiempo de un municipio espanol. La interfaz es intuitiva: se escribe el nombre del municipio, se selecciona de una lista y se pulsa "Consultar tiempo".

Funcionalidades principales

- Busqueda meteorologica por municipio
- Visualizacion de prediccion diaria (varios dias)
- Datos mostrados: estado del cielo, temperaturas, precipitacion, viento y fecha
- Estados visuales de carga, error y sin resultados
- Diseno adaptable a movil y escritorio

- Iconos meteorologicos segun el estado del cielo
 - Modo oscuro con preferencia guardada
-

2. Objetivos

Los objetivos de este proyecto, alineados con el enunciado de DWEC, son:

| Objetivo | Como se ha cumplido | |-----|-----| | Desarrollar una app React con componentes | Componentes funcionales: Header, SearchForm, WeatherCard, Loading, etc. | | Consumir una API REST de forma asincrona | fetch desde React al backend Express | | Aplicar arquitectura frontend-backend desacoplada | React solo habla con Express; Express habla con AEMET | | Implementar backend en Node + Express | Servidor en `backend/server.js` con rutas propias | | Gestionar estados de carga, error y sin datos | Componentes Loading, ErrorMessage y NoResults | | Aplicar usabilidad y diseno responsive | CSS con media queries y layout adaptable | | Documentar el proyecto | Esta memoria y el README del repositorio |

3. Tecnologias utilizadas

Frontend

| Tecnologia | Para que sirve | |-----|-----| | **React** | Libreria para construir la interfaz con componentes reutilizables | | **Vite** | Herramienta de desarrollo rapida para crear y compilar el proyecto React | | **JavaScript (ES6+)** | Lenguaje de programacion (arrow functions, async/await, destructuring) | | **HTML5** | Estructura semantica de la pagina | | **CSS3** | Estilos, variables CSS, media queries para responsive |

Backend

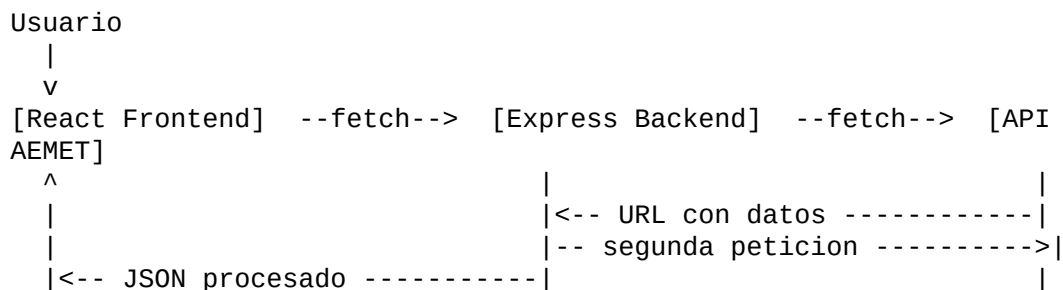
| Tecnologia | Para que sirve | |-----|-----| | **Node.js** | Entorno de ejecucion de JavaScript en el servidor | | **Express** | Framework minimalista para crear la API REST | | **dotenv** | Gestion de variables de entorno (clave AEMET) | | **cors** | Permite que el frontend (puerto 5173) llame al backend (puerto 3000) |

APIs y herramientas

| Tecnologia | Para que sirve | |-----|-----| | **API OpenData AEMET** | Fuente oficial de datos meteorologicos de Espana | | **Git / GitHub** | Control de versiones y alojamiento del codigo | | **npm** | Gestor de paquetes para instalar dependencias |

4. Arquitectura del sistema

Diagrama de flujo



Por que el frontend NO llama a AEMET directamente

Esta es una decision de arquitectura **obligatoria** segun el enunciado:

- 1 **Seguridad:** La clave API de AEMET no debe exponerse en el codigo del navegador. Cualquier usuario podria verla en las herramientas de desarrollo.
- 2 **Control:** El backend puede validar peticiones, transformar datos y devolver solo lo necesario.
- 3 **Mantenimiento:** Si cambia la API de AEMET, solo hay que modificar el backend, no toda la aplicacion React.

Flujo de datos al buscar un municipio

- 1 El usuario selecciona un municipio y pulsa "Consultar tiempo"
 - 2 React llama a GET /api/tiempo/municipio/30016 (ejemplo: Cartagena)
 - 3 Express valida el codigo y llama a AEMET con la clave API
 - 4 AEMET devuelve una URL temporal con los datos reales
 - 5 Express hace una segunda peticion a esa URL
 - 6 Express simplifica el JSON y lo devuelve a React
 - 7 React muestra las tarjetas meteorologicas
-

5. Estructura del repositorio

```
proyecto-react-final-jfmc/
.
... backend/                                # Servidor Node.js + Express
.  ... services/
.    ... aemetService.js                    # Logica de comunicacion con AEMET
.    ... server.js                          # Rutas y arranque del servidor
.    ... package.json                       # Dependencias del backend
.    ... .env.example                       # Plantilla de variables de entorno
.    ... .gitignore
.
... frontend/                                # Aplicacion React
.  ... src/
.    ... components/                        # Componentes de la interfaz
.      ... Header.jsx
.      ... SearchForm.jsx
.      ... WeatherCard.jsx
.      ... Loading.jsx
.      ... ErrorMessage.jsx
.      ... NoResults.jsx
.    ... hooks/
.      ... useTheme.js                      # Hook para modo oscuro
.      ... services/
.        ... api.js                         # Funciones fetch al backend
.        ... utils/
.          ... weatherIcons.js              # Mapeo estado cielo -> icono
.          ... App.jsx                      # Componente principal
.          ... App.css
.          ... index.css                    # Variables CSS y tema
```

· · ... main.jsx	# Punto de entrada
· ... vite.config.js	# Configuración Vite + proxy
· ... package.json	
·	
... docs/	# Documentación del proyecto
· ... enunciado.pdf	
· ... Especificaciones.pdf	
· ... memoria.md	# Este documento
· ... memoria.pdf	# Versión PDF para entrega
·	
... package.json	# Scripts de ayuda en la raíz
... README.md	# Guía rápida
... .gitignore	

6. Backend: servidor Express y AEMET

Endpoints disponibles

GET /

Devuelve información básica de la API y lista de endpoints.

GET /api/municipios

- Consulta el maestro de municipios de AEMET (/maestro/municipios)
- Guarda el resultado en **cache en memoria** para no repetir la petición
- Devuelve un array con: código, nombre, latitud, longitud

Respuesta de ejemplo:

```
{
  "success": true,
  "total": 8132,
  "data": [
    { "codigo": "30016", "nombre": "Cartagena", "latitud":
"37.6092", "longitud": "-0.9834" }
  ]
}
```

GET /api/tiempo/municipio/:codigo

- Valida que el código tenga 5 dígitos
- Consulta predicción diaria: /prediccion/especifica/municipio/diaria/{codigo}
- Transforma el JSON complejo de AEMET a un formato simple para el frontend

Respuesta de ejemplo:

```
{
  "success": true,
  "data": {
    "nombre": "Cartagena",
    "provincia": "Murcia",
    "elaborado": "2026-06-19T08:00:00",
  }
}
```

```

    "dias": [
      {
        "fecha": "2026-06-19",
        "estadoCielo": "Despejado",
        "temperaturaMax": 32,
        "temperaturaMin": 22,
        "probPrecipitacion": 0,
        "vientoDireccion": "E",
        "vientoVelocidad": 15
      }
    ]
  }
}

```

Servicio AEMET (aemetService.js)

Este archivo centraliza toda la comunicacion con AEMET:

- `fetchAemet(endpoint)`: Funcion generica que implementa el patron de dos peticiones
- `getMunicipios()`: Obtiene y cachea la lista de municipios
- `getPrediccionMunicipio(codigo)`: Obtiene y transforma la prediccion

Patron de dos peticiones de AEMET

AEMET no devuelve los datos directamente. El flujo es:

- 1 Primera peticion:
`https://opendata.aemet.es/opendata/api/...?api_key=CLAVE`
- 2 Respuesta: `{ "estado": 200, "datos": "https://opendata.aemet.es/opendata/sh/XXXXXX" }`
- 3 Segunda peticion a la URL de datos
- 4 Ahi estan los datos meteorologicos reales en JSON

Esta URL temporal expira en unos minutos, por eso hay que usarla inmediatamente.

Variables de entorno

Archivo `backend/.env` (no se sube a Git):

```

PORT=3000
AEMET_API_KEY=tu_clave_personal

```

7. Frontend: aplicacion React

Componentes

| Componente | Responsabilidad | |-----|-----| | `App.jsx` | Estado global: datos, carga, error. Orquesta la busqueda | | `Header.jsx` | Titulo de la app y boton de modo oscuro | | `SearchForm.jsx` | Carga municipios, filtro por nombre, selector y validacion | | `WeatherCard.jsx` | Muestra tarjetas con la prediccion de cada dia | | `Loading.jsx` | Indicador visual de carga con spinner | | `ErrorMessage.jsx` | Mensaje amigable cuando algo falla | | `NoResults.jsx` | Mensaje cuando no hay datos |

Hooks utilizados

- **useState:** Gestionar estado (datos del tiempo, carga, errores, tema)
- **useEffect:** Cargar lista de municipios al montar SearchForm
- **useTheme (custom):** Gestionar modo oscuro y persistir en localStorage

Servicio API (api.js)

Funciones que encapsulan las llamadas fetch:

```
getMunicipios() // GET /api/municipios
getTiempoMunicipio(codigo) // GET /api/tiempo/municipio/:codigo
```

Ambas funciones comprueban `response.ok` y `data.success` antes de devolver datos.

Proxy de desarrollo

En `vite.config.js` se configura un proxy para que las peticiones a `/api` se redirijan al backend en `localhost:3000`. Esto evita problemas de CORS durante el desarrollo.

8. Instalacion y ejecucion

Requisitos previos

- Node.js 18 o superior instalado
- Cuenta en OpenData AEMET con clave API

Pasos de instalacion

```
# 1. Clonar el repositorio
git clone
https://github.com/josecostasdaw/proyecto-react-final-jfmc.git
cd proyecto-react-final-jfmc
# 2. Instalar dependencias
npm run install:all
# 3. Configurar clave AEMET
cd backend
cp .env.example .env
# Editar .env con tu AEMET_API_KEY real
```

Arrancar la aplicacion

Terminal 1 - Backend:

```
npm run dev:backend
```

Terminal 2 - Frontend:

```
npm run dev:frontend
```

Abrir el navegador en `http://localhost:5173`

Compilar para produccion

```
npm run build:frontend
```

Los archivos compilados se generan en `frontend/dist/`.

9. Decisiones tecnicas

Por que Vite en lugar de Create React App

Vite es mas rapido en desarrollo (arranque instantaneo) y es el estandar actual para proyectos React nuevos. Create React App esta practicamente abandonado.

Por que busqueda por municipio

El enunciado permite elegir el tipo de busqueda. Se eligio municipio porque:

- Es la consulta mas comun para usuarios ("que tiempo hace en mi ciudad")
- La API de AEMET tiene endpoints especificos por codigo de municipio
- Permite implementar un selector con filtro por nombre

Por que cache en memoria para municipios

La lista de municipios tiene mas de 8000 entradas y no cambia frecuentemente. Guardarla en memoria del servidor evita llamar a AEMET en cada visita, reduciendo tiempo de espera y peticiones a la API.

Por que CSS plano sin librerias UI

Para un proyecto de aprendizaje, usar CSS directo con variables permite entender mejor el diseno responsive y el modo oscuro sin depender de frameworks como Bootstrap o Tailwind.

Por que emojis como iconos meteorologicos

Son ligeros (sin dependencias extra), funcionan en todos los navegadores y son suficientes para un proyecto educativo. Se mapean segun el texto del estado del cielo.

Por que transformar el JSON de AEMET en el backend

El JSON de AEMET es muy complejo y anidado. Simplificarlo en el backend hace que el frontend sea mas sencillo de entender y mantener, que es el objetivo pedagogico del modulo.

10. Mejoras implementadas

Ademas de los requisitos minimos, se han anadido dos mejoras opcionales del enunciado:

Iconos meteorologicos

El archivo `weatherIcons.js` mapea el estado del cielo a un emoji:

| Estado | Icono | |-----|-----| Despejado | ☀️ | Poco nuboso | ☁️ | Intervalos nubosos | ⛅️ | Nuboso / lluvia | 🌧️ | Nieve | ❄️ | Tormenta | 🌩️ | Niebla / calima | 🌫️ |

Modo oscuro

- Variables CSS en `:root` (tema claro) y `[data-theme="dark"]` (tema oscuro)
 - Boton toggle en la cabecera (· / ··)
 - Preferencia guardada en `localStorage` con la clave `meteo-theme`
 - Hook personalizado `useTheme` para encapsular la logica
-

11. Gestion de errores

En el backend

| Situacion | Respuesta | |-----|-----| |Codigo de municipio invalido | HTTP 400 con mensaje claro | | Clave AEMET no configurada | HTTP 500, mensaje generico | | AEMET no responde o devuelve error | HTTP 500, sin detalles tecnicos | | Sin datos para el municipio | HTTP 404 | | Ruta no encontrada | HTTP 404 |

Los errores tecnicos se registran con `console.error` en el servidor, pero al usuario solo se le muestra un mensaje comprensible.

En el frontend

| Situacion | Componente | |-----|-----| |Cargando datos | `Loading` con spinner | | Error de red o del servidor | `ErrorMessage` | | Busqueda sin resultados | `NoResults` | | Municipio no seleccionado | Validacion en `SearchForm` | | Lista de municipios no carga | `ErrorMessage` en el formulario |

El usuario **nunca** ve stack traces, codigos HTTP ni JSON crudo de errores.

12. Dificultades encontradas y soluciones

La API de AEMET devuelve una URL, no los datos directamente

Problema: Al hacer la primera peticion a AEMET, la respuesta no contiene datos meteorologicos, sino una URL temporal.

Solucion: Implementar el patron de dos peticiones en `fetchAemet()`: primero obtener la URL, luego hacer fetch a esa URL.

El JSON de AEMET es muy complejo

Problema: La respuesta de prediccion tiene muchos campos anidados (periodos, arrays de estado del cielo, etc.).

Solucion: Crear funciones de transformacion en el backend (`obtenerEstadoCielo`, `obtenerTemperatura`, etc.) que extraen solo los datos necesarios y los devuelven en un formato plano.

Mas de 8000 municipios en el selector

Problema: Cargar todos los municipios en un `<select>` sin filtro seria lento e incomodo.

Solucion: Campo de texto para filtrar por nombre + select que muestra solo los 100 primeros resultados del filtro.

Los codigos de estado del cielo son numericos

Problema: AEMET devuelve codigos como "11", "12", "24" en lugar de texto descriptivo.

Solucion: Tabla de mapeo ESTADOS_CIELO en el backend que traduce codigos a descripciones en espanol.

CORS en desarrollo

Problema: El frontend (puerto 5173) y el backend (puerto 3000) son origenes diferentes.

Solucion: Proxy en Vite (/api -> localhost:3000) y middleware CORS en Express.

13. Conclusiones

Que he aprendido

- Como estructurar una aplicacion web moderna con **arquitectura cliente-servidor**
- A consumir una **API REST real** (AEMET) de forma asincrona con `fetch` y `async/await`
- A organizar un proyecto React en **componentes** con responsabilidades claras
- A usar **hooks** (`useState`, `useEffect`) y crear hooks personalizados (`useTheme`)
- La importancia de **no exponer claves API** en el frontend
- A gestionar **estados de UI** (carga, error, sin datos) para una buena experiencia de usuario
- A aplicar **diseño responsive** con CSS y media queries
- A usar **Git** con ramas y commits organizados por fases

Que mejoraria con mas tiempo

- Anadir busqueda por provincia como segundo tipo de consulta
- Implementar prediccion por horas ademas de la diaria
- Anadir historial de busquedas recientes con `localStorage`
- Mejorar la accesibilidad (navegacion por teclado, lectores de pantalla)
- Anadir tests automatizados (Jest para backend, React Testing Library para frontend)
- Desplegar la aplicacion en un servidor (Render, Vercel, etc.)

Valoracion personal

Este proyecto ha sido una oportunidad para integrar conocimientos de distintas areas (JavaScript, React, Node.js, APIs, Git) en una aplicacion funcional y completa. La mayor dificultad ha sido entender el formato de la API de AEMET, pero una vez comprendido el patron de dos peticiones, el resto del desarrollo ha sido mas directo.

14. Referencias

- API OpenData AEMET
 - Documentacion de React
 - Documentacion de Express
 - Documentacion de Vite
 - Fetch API - MDN
 - Repositorio del proyecto
-

Documento generado como memoria del proyecto integrador DWEC - Aplicacion Meteorologica con React y AEMET.